

Informe Técnico: Monitoreo Automatizado en Ubuntu Desktop

Implementación de Scripts Bash Independientes y Automatización Periódica con Crontab

Alumno: Piña Ríos Fernando Emmanuel

Carrera: Ingeniería en Sistemas Computacionales

Institución: Instituto Tecnológico de Mazatlán (ITMAZ)

Materia: Sistemas Operativos / Entornos Linux

1. Planteamiento del Problema

En entornos de desarrollo y estudio bajo Ubuntu Desktop, el flujo de trabajo típico de un desarrollador implica la ejecución simultánea de múltiples aplicaciones intensivas: navegadores con docenas de pestañas de documentación, entornos integrados de desarrollo (IDE), emuladores, servidores de bases de datos locales y terminales concurrentes. Esta carga sostenida genera dos riesgos operacionales graves:

- **Saturación de Memoria RAM:** Cuando la memoria física se agota, el sistema activa intensivamente la memoria swap (paginación en disco), degradando drásticamente el rendimiento hasta provocar el congelamiento del entorno gráfico o el forzado del *OOM Killer (Out of Memory Killer)* del kernel Linux.
- **Agotamiento de la Partición Raíz (/):** La acumulación inadvertida de paquetes de compilación temporales, cachés de navegadores, logs del sistema y descargas masivas puede colapsar el almacenamiento, impidiendo la escritura de nuevos ficheros del sistema.

Para mitigar esto, se requiere un mecanismo desatendido que registre la salud de ambos recursos de forma periódica e independiente, almacenando las métricas en la carpeta personal del usuario para su posterior auditoría.

2. Objetivos de la Práctica

- Desarrollar dos rutinas independientes en Bash: una dedicada al diagnóstico de la memoria RAM y otra al almacenamiento del disco raíz.
- Configurar el demonio de planificación cron para disparar la ejecución automática en segundo plano cada 2 minutos.
- Almacenar el historial cronológico estructurado en archivos .log ubicados dentro del directorio del usuario (\$HOME).
- Garantizar la modularidad, control de versiones y trazabilidad de los scripts mediante Git y GitHub.

3. Fundamento Técnico

3.1. Arquitectura del Demonio Cron

El servicio cron es el planificador de tareas estándar en sistemas tipo UNIX. Opera como un demonio en background (crond o cron) que despierta cada minuto para examinar los archivos de programación (crontabs). La estructura de cada entrada se compone de cinco campos temporales seguidos de la ruta absoluta del ejecutable:

```

|----- Minuto (0 - 59) o intervalo */2
| |----- Hora (0 - 23)
| | |----- Día del mes (1 - 31)
| | | |----- Mes (1 - 12)
| | | | |----- Día de la semana (0 - 6) (Domingo = 0 o 7)
| | | | |
| | | | | * * * * * comando_a_ejecutar

```

La expresión `*/2 * * * *` define un paso aritmético que instruye al planificador a despachar la tarea cada dos minutos con exactitud de reloj.

3.2. Herramientas de Extracción de Métricas

- **Comando `free -h`**: Lee el pseudo-sistema de ficheros `/proc/meminfo` y entrega la memoria física y swap en múltiplos de potencias de 1024 (KiB, MiB, GiB), distinguiendo entre memoria libre, usada y la memoria en buffers/caché que puede liberarse inmediatamente.
- **Comando `df -h /`**: Consulta las llamadas del sistema a `statfs` para inspeccionar el sistema de archivos montado en la raíz (`/`), extrayendo el tamaño global, espacio ocupado, disponibilidad real y porcentaje porcentual de uso.

4. Metodología de Implementación Paso a Paso

Paso #1: Creación de los Scripts Independientes

En el directorio de inicio del usuario (`$HOME`), se crean los ficheros correspondientes sin acoplamiento de código:

```

touch ~/registro_ram.sh
touch ~/registro_almacenamiento.sh

```

Paso #2: Implementación del Script de Memoria RAM

El script añade encabezados cronológicos y la salida de `free -h` al archivo `historial_ram.log` utilizando redirección de flujo (`>>`) para preservar los registros anteriores:

```

cat << 'EOF' > ~/registro_ram.sh
#!/bin/bash
LOG_FILE="$HOME/historial_ram.log"

echo "=====" >> "$LOG_FILE"
echo "Registro RAM: $(date '+%Y-%m-%d %H:%M:%S')" >> "$LOG_FILE"
free -h >> "$LOG_FILE"
echo "" >> "$LOG_FILE"
EOF

```

Paso #3: Implementación del Script de Almacenamiento

De forma análoga, el script captura el consumo de la partición principal en `historial_almacenamiento.log`:

```
cat << 'EOF' > ~/registro_almacenamiento.sh
#!/bin/bash
LOG_FILE="$HOME/historial_almacenamiento.log"

echo "=====" >> "$LOG_FILE"
echo "Registro Almacenamiento: $(date '+%Y-%m-%d %H:%M:%S')" >> "$LOG_FILE"
df -h / >> "$LOG_FILE"
echo "" >> "$LOG_FILE"
EOF
```

Paso #4: Asignación de Permisos de Ejecución

Se configuran los bits de modo POSIX correspondientes (u+x) para habilitar su invocación desatendida por el sistema:

```
chmod +x ~/registro_ram.sh
chmod +x ~/registro_almacenamiento.sh
```

Paso #5: Programación Automatizada en Crontab

Se incorporan ambas tareas al crontab del usuario conservando las entradas preexistentes:

```
(crontab -l 2>/dev/null; echo "*/2 * * * * $HOME/registro_ram.sh") | crontab -
(crontab -l 2>/dev/null; echo "*/2 * * * * $HOME/registro_almacenamiento.sh") | crontab -
```

Paso #6: Control de Versiones y Publicación en GitHub

```
git init
git add registro_ram.sh registro_almacenamiento.sh
git commit -m "Implementacion de scripts independientes de monitoreo RAM y disco"
git branch -M main
git remote add origin https://github.com/TU_USUARIO/TU_REPOSITORIO.git
git push -u origin main
```

5. Conclusión

La solución implementada cumple a cabalidad los principios de modularidad y robustez exigidos en entornos Linux avanzados. Al separar la recolección de memoria y almacenamiento en procesos aislados, un eventual error en el subsistema de discos no interrumpe el seguimiento del rendimiento de la memoria principal. La automatización vía cron garantiza una vigilancia continua sin demandar intervención manual ni consumir ciclos de CPU apreciables.